

PRACTICAL 2

Problem Definition

To programmatically interface with diverse file-based, API-driven, and relational database source environments using Python. The practical focuses on generating simulated customer and transaction data, discovering schemas, identifying data quality issues such as null values, duplicates, and format divergence, and applying appropriate ingestion rules.

Objectives

- To generate synthetic customer and transaction data using Faker.
- To ingest data from CSV, JSON, REST API, text files, and SQLite databases.
- To automatically discover and profile incoming data schemas.
- To identify missing values, duplicates, invalid formats, and invalid values.
- To define operational ingestion criteria for accepting or rejecting records.
- To store clean data in a relational database and perform SQL queries.

```
In [1]: !pip install faker pandas requests
```

```
Requirement already satisfied: faker in c:\users\digisha\anaconda3\lib\site-packages (40.35.0)
Requirement already satisfied: pandas in c:\users\digisha\anaconda3\lib\site-packages (2.3.3)
Requirement already satisfied: requests in c:\users\digisha\anaconda3\lib\site-packages (2.32.5)
Requirement already satisfied: tzdata in c:\users\digisha\anaconda3\lib\site-packages (from faker) (2025.2)
Requirement already satisfied: numpy>=1.26.0 in c:\users\digisha\anaconda3\lib\site-packages (from pandas) (2.3.5)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\digisha\anaconda3\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\digisha\anaconda3\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\digisha\anaconda3\lib\site-packages (from requests) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in c:\users\digisha\anaconda3\lib\site-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\digisha\anaconda3\lib\site-packages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\digisha\anaconda3\lib\site-packages (from requests) (2025.11.12)
Requirement already satisfied: six>=1.5 in c:\users\digisha\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

```
In [2]: # =====
# STEP 1: IMPORT REQUIRED LIBRARIES
# =====

import os
import json
```

```
import re
import requests
import pandas as pd

from faker import Faker
from datetime import datetime

print("All required libraries imported successfully.")
```

All required libraries imported successfully.

```
In [3]: # =====
# STEP 2: CREATE PROJECT DIRECTORIES
# =====

os.makedirs("data", exist_ok=True)
os.makedirs("output", exist_ok=True)

print("Project directories created successfully.")
print("data/")
print("output/")
```

Project directories created successfully.
data/
output/

```
In [4]: # =====
# STEP 3: INITIALIZE FAKER
# =====

fake = Faker()

# Setting seed makes generated results reproducible
Faker.seed(42)
random.seed(42)

print("Faker initialized successfully.")
```

Faker initialized successfully.

```
In [5]: # =====
# STEP 4: GENERATE SYNTHETIC CUSTOMER PROFILE DATA
# =====

customers = []

for i in range(1, 501):

    customer = {
        "Customer ID": i,
        "Name": fake.name(),
        "Age": random.randint(18, 80),
        "Gender": random.choice(["Male", "Female"]),
        "Email": fake.email(),
        "Phone": fake.phone_number(),
        "City": fake.city(),
        "Purchase Amount (USD)": round(random.uniform(10, 2000), 2),
        "Review Rating": round(random.uniform(1, 5), 1),
        "Category": random.choice([
            "Clothing",
            "Electronics",
```

```

        "Footwear",
        "Accessories",
        "Home"
    ]),
    "Signup Date":
        fake.date_between( start_date=
            "-3y",
            end_date="today"
        ).strftime("%Y-%m-%d")
    }

    customers.append(customer)

customer_df = pd.DataFrame(customers)

print("Customer dataset generated successfully.")
print("Total records:", len(customer_df))

```

Customer dataset generated successfully.
Total records: 500

Out[5]:

	Customer ID	Name	Age	Gender	Email	Phone	
0	1	Allison Hill	58	Male	donaldgarcia@example.net	+1-219-560-0133	Robins
1	2	Abigail Shaffer	26	Male	jpeterson@example.org	001-851-316-1559x40781	Mar
2	3	Benjamin Stanley	55	Female	tracie31@example.com	575-425-5341x928	Gr
3	4	Sarah Parsons	50	Male	lydiatrujillo@example.com	513-695-3767x242	Hu
4	5	Zachary Hicks	44	Male	camposmichelle@example.org	+1-326-691-6697x8480	Danielc

```

In [6]: print("Dataset Shape:", customer_df.shape)

        print("\nNumber of Rows:", customer_df.shape[0])
        print("Number of Columns:", customer_df.shape[1])

```

Dataset Shape: (500, 11)

Number of Rows: 500

Number of Columns: 11

```

In [7]: # =====
        # STEP 5: INTRODUCE DATA QUALITY VIOLATIONS
        # =====

        # NULL ENTRY LEAKS
        customer_df.loc[5, "Email"] = None

```

```

customer_df.loc[10, "Name"] = None
customer_df.loc[15, "Gender"] = None
customer_df.loc[20, "Category"] = None

# FORMAT DIVERGENCE
customer_df.loc[25, "Email"] = "invalid-email"
customer_df.loc[30, "Email"] = "abc@"
customer_df.loc[35, "Signup Date"] = "wrong-date"

# INVALID BUSINESS VALUES
customer_df.loc[40, "Age"] = 150
customer_df.loc[45, "Age"] = -5
customer_df.loc[50, "Purchase Amount (USD)"] = -500
customer_df.loc[55, "Review Rating"] = 7

# Add duplicate record
duplicate_row = customer_df.iloc[[0]].copy()

customer_df = pd.concat(
    [customer_df, duplicate_row],
    ignore_index=True
)

print("Data-quality violations introduced successfully.")
print("Total records after adding duplicate:", len(customer_df))

```

Data-quality violations introduced successfully.
Total records after adding duplicate: 501

```

In [8]: # =====
# STEP 6: SAVE STRUCTURED DATA TO CSV
# =====

customer_df.to_csv(
    "data/customers.csv",
    index=False
)

print("Customer data saved successfully.")
print("File: data/customers.csv")

```

Customer data saved successfully.
File: data/customers.csv

```

In [9]: # =====
# STEP 7: INGEST FILE-BASED CSV SOURCE
# =====

df = pd.read_csv("data/customers.csv")

print("CSV file loaded successfully.")

df.head()

```

CSV file loaded successfully.

Out[9]:

	Customer ID	Name	Age	Gender	Email	Phone	
0	1	Allison Hill	58	Male	donaldgarcia@example.net	+1-219-560-0133	Robinsons
1	2	Abigail Shaffer	26	Male	jpetererson@example.org	001-851-316-1559x40781	Maria
2	3	Benjamin Stanley	55	Female	tracie31@example.com	575-425-5341x928	Gray
3	4	Sarah Parsons	50	Male	lydiatrujillo@example.com	513-695-3767x242	Hurst
4	5	Zachary Hicks	44	Male	camposmichelle@example.org	+1-326-691-6697x8480	Danielche

```
In [10]: print("Dataset Shape:", df.shape)

print("\nColumn Names:")
print(df.columns.tolist())

print("\nFirst 5 Records:")
display(df.head())
```

Dataset Shape: (501, 11)

Column Names:

['Customer ID', 'Name', 'Age', 'Gender', 'Email', 'Phone', 'City', 'Purchase Amount (USD)', 'Review Rating', 'Category', 'Signup Date']

First 5 Records:

	Customer ID	Name	Age	Gender	Email	Phone	
0	1	Allison Hill	58	Male	donaldgarcia@example.net	+1-219-560-0133	Robinsons
1	2	Abigail Shaffer	26	Male	jpetererson@example.org	001-851-316-1559x40781	Maria
2	3	Benjamin Stanley	55	Female	tracie31@example.com	575-425-5341x928	Gray
3	4	Sarah Parsons	50	Male	lydiatrujillo@example.com	513-695-3767x242	Hurst
4	5	Zachary Hicks	44	Male	camposmichelle@example.org	+1-326-691-6697x8480	Danielche

```

In [11]: # =====
# STEP 8: AUTOMATIC SCHEMA DISCOVERY
# =====

print("AUTOMATIC SCHEMA DISCOVERY")
print("=" * 60)

schema_data = []

for column in df.columns:

    schema_data.append({ "Column": column,
                        "Data Type": str(df[column].dtype),
                        "Non-Null Count": int(df[column].notnull().sum()),
                        "Null Count": int(df[column].isnull().sum()),
                        "Unique Values": int(df[column].nunique())
                    })

schema_df = pd.DataFrame(schema_data)

schema_df

```

AUTOMATIC SCHEMA DISCOVERY

=====

```

Out[11]:

```

	Column	Data Type	Non-Null Count	Null Count	Unique Values
0	Customer ID	int64	501	0	500
1	Name	object	500	1	494
2	Age	int64	501	0	65
3	Gender	object	500	1	2
4	Email	object	500	1	498
5	Phone	object	501	0	500
6	City	object	501	0	491
7	Purchase Amount (USD)	float64	501	0	500
8	Review Rating	float64	501	0	42
9	Category	object	500	1	5
10	Signup Date	object	501	0	409

```

In [12]: print("PANDAS DATASET INFORMATION")
print("=" * 60)

df.info()

```

PANDAS DATASET INFORMATION

```
=====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 501 entries, 0 to 500
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Customer ID           501 non-null   int64
1   Name                   500 non-null   object
2   Age                    501 non-null   int64
3   Gender                 500 non-null   object
4   Email                  500 non-null   object
5   Phone                  501 non-null   object
6   City                   501 non-null   object
7   Purchase Amount (USD) 501 non-null   float64
8   Review Rating          501 non-null   float64
9   Category               500 non-null   object
10  Signup Date            501 non-null   object
dtypes: float64(2), int64(2), object(7)
memory usage: 43.2+ KB
```

```
In [13]: # =====
# STEP 9: DATA PROFILING
# =====

print("STATISTICAL DATA PROFILE")
print("=" * 60)

display(df.describe(include="all"))
```

STATISTICAL DATA PROFILE

=====

	Customer ID	Name	Age	Gender	Email	Phone	Cit
count	501.000000	500	501.000000	500	500	501	50
unique	NaN	494	NaN	2	498	500	49
top	NaN	Allison Hill	NaN	Male	donaldgarcia@example.net	+1-219-560-0133	Sout Michae
freq	NaN	2	NaN	258	2	2	
mean	250.001996	NaN	48.043912	NaN	NaN	NaN	Na
std	144.767061	NaN	18.418959	NaN	NaN	NaN	Na
min	1.000000	NaN	-5.000000	NaN	NaN	NaN	Na
25%	125.000000	NaN	33.000000	NaN	NaN	NaN	Na
50%	250.000000	NaN	47.000000	NaN	NaN	NaN	Na
75%	375.000000	NaN	63.000000	NaN	NaN	NaN	Na
max	500.000000	NaN	150.000000	NaN	NaN	NaN	Na

```
In [14]: print("MISSING VALUE ANALYSIS")
print("=" * 60)

missing_values = df.isnull().sum()

missing_percentage = (
    df.isnull().sum() / len(df)
) * 100

missing_report = pd.DataFrame({
    "Missing Values": missing_values,
    "Missing Percentage": missing_percentage
})

display(missing_report)
```

MISSING VALUE ANALYSIS

=====

	Missing Values	Missing Percentage
Customer ID	0	0.000000
Name	1	0.199601
Age	0	0.000000
Gender	1	0.199601
Email	1	0.199601
Phone	0	0.000000
City	0	0.000000
Purchase Amount (USD)	0	0.000000
Review Rating	0	0.000000
Category	1	0.199601
Signup Date	0	0.000000

```
In [15]: null_records = df[
    df.isnull().any(axis=1)
]

print("Records containing NULL values:")
print("Number of affected records:", len(null_records))

display(null_records)
```

Records containing NULL values:

Number of affected records: 4

	Customer ID	Name	Age	Gender	Email	Phone	
5	6	Troy Liu	66	Male	NaN	+1-948-293-2528x8095	Natash
10	11	NaN	60	Male	smoore@example.org	4982050097	Jaco
15	16	Norman Cook	32	NaN	vpham@example.org	521-260-7337x543	South
20	21	Shannon McClure	26	Female	zrobinson@example.net	870.983.1727x88957	Mic

```
In [16]: # =====
# DUPLICATE DETECTION
# =====

duplicate_count = df.duplicated().sum()

print("Number of duplicate records:", duplicate_count)

duplicate_records = df[
    df.duplicated(keep=False)
]

display(duplicate_records)
```

Number of duplicate records: 1

	Customer ID	Name	Age	Gender	Email	Phone	City	Pu A
0	1	Allison Hill	58	Male	donaldgarcia@example.net	+1-219-560-0133	Robinsonshire	
500	1	Allison Hill	58	Male	donaldgarcia@example.net	+1-219-560-0133	Robinsonshire	

```
In [17]: # =====
# STEP 10: FORMAT DIVERGENCE DETECTION
# =====

email_pattern = r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'

invalid_email_mask = (
    df["Email"].notna()
    & ~df["Email"].astype(str).str.match(email_pattern)
)
```

```
invalid_emails = df[invalid_email_mask]

print("Invalid Email Records:", len(invalid_emails))

display(
    invalid_emails[
        ["Customer ID", "Name", "Email"]
    ]
)
```

Invalid Email Records: 2

	Customer ID	Name	Email
25	26	Travis Marshall	invalid-email
30	31	Katherine Salas	abc@

```
In [18]: # Convert signup date to datetime

df["Parsed Signup Date"] =
    pd.to_datetime( df["Signup Date"],
        errors="coerce"
    )

invalid_date_mask = (
    df["Signup Date"].notna()
    & df["Parsed Signup Date"].isna()
)

invalid_dates = df[invalid_date_mask]

print("Invalid Date Records:", len(invalid_dates))

display(
    invalid_dates[
        ["Customer ID", "Name", "Signup Date"]
    ]
)
```

Invalid Date Records: 1

	Customer ID	Name	Signup Date
35	36	Jennifer Hodges	wrong-date

```
In [19]: invalid_age = df[
    (df["Age"] < 18)
    | (df["Age"] > 100)
]

print("Invalid Age Records:", len(invalid_age))

display(
    invalid_age[
        ["Customer ID", "Name", "Age"]
    ]
)
```

Invalid Age Records: 2

	Customer ID	Name	Age
40	41	Paige Padilla	150
45	46	Connie Brown	-5

```
In [20]: invalid_purchase = df[
    df["Purchase Amount (USD)"] <= 0
]

print(
    "Invalid Purchase Amount Records:",
    len(invalid_purchase)
)

display(
    invalid_purchase[
        [
            "Customer ID",
            "Name",
            "Purchase Amount (USD)"
        ]
    ]
)
```

Invalid Purchase Amount Records: 1

	Customer ID	Name	Purchase Amount (USD)
50	51	Patricia Le	-500.0

```
In [21]: invalid_rating = df[
    ~df["Review Rating"].between(1, 5)
]

print(
    "Invalid Review Rating Records:",
    len(invalid_rating)
)

display(
    invalid_rating[
        [
            "Customer ID",
            "Name",
            "Review Rating"
        ]
    ]
)
```

Invalid Review Rating Records: 1

	Customer ID	Name	Review Rating
55	56	James Padilla	7.0

```
In [22]: # =====
# STEP 11: GENERATE SEMI-STRUCTURED TRANSACTION LOGS
# =====
```

```
transactions = []

for i in range(1, 201):

    transaction = {

        "transaction_id": f"TXN{i:04d}",

        "customer": {
            "customer_id": random.randint(1, 500),
            "name": fake.name(),
            "location": {
                "city": fake.city(),
                "country": fake.country()
            }
        },

        "purchase": {
            "item": fake.word(),
            "category":
                random.choice([ "Electr
onics",
"Clothing",
"Footwear",
"Home",
"Accessories"
]),
            "amount": round(
                random.uniform(10, 2000),
                2
            ),
            "quantity": random.randint(1, 5)
        },

        "payment": {
            "method":
                random.choice([ "UPI",
"Credit Card",
"Debit Card",
"Cash"
]),
            "status":
                random.choice([ "Success",
"Success",
"Success",
"Failed"
])
        },

        "timestamp": fake.date_time_this_year().isoformat()
    }

    transactions.append(transaction)

print(
    "Generated",
    len(transactions),
    "transaction records."
)
```

Generated 200 transaction records.

```
In [23]: print(
    json.dumps(
        transactions[0],
        indent=4
    )
)

{
  "transaction_id": "TXN0001",
  "customer": {
    "customer_id": 428,
    "name": "Kristina Henry",
    "location": {
      "city": "Ginaside",
      "country": "United Arab Emirates"
    }
  },
  "purchase": {
    "item": "learn",
    "category": "Home",
    "amount": 1357.3,
    "quantity": 5
  },
  "payment": {
    "method": "Debit Card",
    "status": "Failed"
  },
  "timestamp": "2026-05-16T05:16:38"
}
```

```
In [24]: with open(
    "data/transactions.json",
    "w",
    encoding="utf-8"
) as file:

    json.dump(
        transactions,
        file,
        indent=4
    )

    print(
        "Nested transaction JSON saved successfully."
    )
```

Nested transaction JSON saved successfully.

```
In [25]: # =====
# STEP 12: INGEST SEMI-STRUCTURED JSON
# =====

with open(
    "data/transactions.json",
    "r",
    encoding="utf-8"
) as file:
```

```
transaction_data = json.load(file)

print(
    "Number of transactions loaded:",
    len(transaction_data)
)
```

Number of transactions loaded: 200

```
In [26]: transaction_df =
         pd.json_normalize(transaction_data)

print(
    "Flattened Transaction Dataset Shape:",
    transaction_df.shape
)

display(transaction_df.head())
```

Flattened Transaction Dataset Shape: (200, 12)

	transaction_id	timestamp	customer.customer_id	customer.name	customer.location.city
0	TXN0001	2026-05-16T05:16:38	428	Kristina Henry	Ginasid
1	TXN0002	2026-03-16T04:06:00	387	Sean Wyatt	Jessicaboroug
2	TXN0003	2026-05-01T13:03:03	51	Christopher Adkins	East Scotthave
3	TXN0004	2026-06-04T16:22:09	497	Thomas Moon	New Joshuavie
4	TXN0005	2026-06-26T12:04:02	117	Michael King	New Madelinefor

```
In [27]: print("TRANSACTION SCHEMA")
         print("=" * 60)

         print(transaction_df.dtypes)
```

TRANSACTION SCHEMA

```
=====
transaction_id      object
timestamp           object
customer.customer_id  int64
customer.name       object
customer.location.city  object
customer.location.country  object
purchase.item       object
purchase.category   object
purchase.amount     float64
purchase.quantity   int64
payment.method      object
payment.status      object
dtype: object
```

```
In [28]: print("Missing Values:")
print(transaction_df.isnull().sum())

print("\nDuplicate Transactions:")
print(transaction_df.duplicated().sum())

print("\nTransaction Statistics:")

display(
    transaction_df.describe(
        include="all"
    )
)
```

Missing Values:

```
transaction_id      0
timestamp           0
customer.customer_id 0
customer.name       0
customer.location.city 0
customer.location.country 0
purchase.item       0
purchase.category   0
purchase.amount     0
purchase.quantity   0
payment.method      0
payment.status      0
dtype: int64
```

Duplicate Transactions:

0

Transaction Statistics:

	transaction_id	timestamp	customer.customer_id	customer.name	customer.location.city
count	200	200	200.000000	200	200
unique	200	200	NaN	200	200
top	TXN0001	2026-05-16T05:16:38	NaN	Kristina Henry	C
freq	1	1	NaN	1	1
mean	NaN	NaN	256.085000	NaN	NaN
std	NaN	NaN	149.901568	NaN	NaN
min	NaN	NaN	1.000000	NaN	NaN
25%	NaN	NaN	135.500000	NaN	NaN
50%	NaN	NaN	269.500000	NaN	NaN
75%	NaN	NaN	381.500000	NaN	NaN
max	NaN	NaN	499.000000	NaN	NaN

```
In [29]: transaction_df.to_csv(
    "output/flattened_transactions.csv",
```

```
        index=False
    )

    print(
        "Flattened transactions saved successfully."
    )
```

Flattened transactions saved successfully.

```
In [30]: # =====
# STEP 13: API DATA INGESTION USING REQUESTS
# =====

api_url = "https://jsonplaceholder.typicode.com/users"

try:

    response =
        requests.get( api_url
            ,
            timeout=10
        )

    print(
        "HTTP Status Code:",
        response.status_code
    )

    if response.status_code == 200:

        api_data = response.json()

        print(
            "API records received:",
            len(api_data)
        )

    else:

        api_data = []

        print(
            "API request was unsuccessful."
        )

except requests.RequestException as e:

    api_data = []

    print(
        "API Request Error:",
        e
    )
```

HTTP Status Code: 200
API records received: 10

```
In [31]: if api_data:

    api_df =
        pd.json_normalize( api_data
    )
```



```

print(
    "API Dataset Shape:",
    api_df.shape
)

display(api_df.head())

else:

    api_df = pd.DataFrame()

    print(
        "No API data available."
    )

```

API Dataset Shape: (10, 15)

	id	name	username	email	phone	website	address.s
0	1	Leanne Graham	Bret	Sincere@april.biz	1-770- 736-8031 x56442	hildegard.org	Kulas
1	2	LIVIA Howell	Antonette	Shanna@melissa.tv	010-692- 6593 x09125	anastasia.net	Victor
2	3	Clementine Bauch	Samantha	Nathan@yesenia.net	1-463- 123-4447	ramiro.info	Exe
3	4	Patricia Lebsack	Karianne	Julianne.OConner@kory.org	493-170- 9623 x156	kale.biz	Hoegel
4	5	Chelsey Dietrich	Kamren	Lucio_Hettinger@annie.ca	(254)954- 1289	demarco.info	Skiles

C  C

```

In [32]: if not api_df.empty:

    print("API SCHEMA")
    print("=" * 60)

    print(api_df.dtypes)

    print("\nMissing Values:")
    print(api_df.isnull().sum())

    print("\nDuplicate Records:")
    print(api_df.duplicated().sum())

```

API SCHEMA

```
=====
id                int64
name              object
username          object
email             object
phone            object
website          object
address.street    object
address.suite     object
address.city      object
address.zipcode   object
address.geo.lat   object
address.geo.lng   object
company.name      object
company.catchPhrase object
company.bs        object
dtype: object
```

Missing Values:

```
id                0
name              0
username          0
email             0
phone            0
website          0
address.street    0
address.suite     0
address.city      0
address.zipcode   0
address.geo.lat   0
address.geo.lng   0
company.name      0
company.catchPhrase 0
company.bs        0
dtype: int64
```

Duplicate Records:

```
0
```

```
In [33]: if not api_df.empty:

         api_df.to_csv(
             "output/api_users.csv",
             index=False
         )

         print(
             "API data saved successfully."
         )
```

API data saved successfully.

```
In [34]: # =====
# STEP 14: CREATE UNSTRUCTURED CONFIGURATION TEXT FILE
# =====

config_text = """
source_system=customer_shopping_system
environment=development
```

```

database=customer_database.db
batch_size=100
retry_count=3
minimum_age=18
maximum_age=100
minimum_rating=1
maximum_rating=5
allow_null_customer_id=false
"""

with open(
    "data/config.txt",
    "w"
) as file:

    file.write(config_text)

print(
    "Configuration text file created successfully."
)

```

Configuration text file created successfully.

```

In [35]: with open(
            "data/config.txt",
            "r"
        ) as file:

            raw_config = file.read()

        print("RAW CONFIGURATION FILE")
        print("=" * 60)

        print(raw_config)

```

RAW CONFIGURATION FILE

=====

```

source_system=customer_shopping_system
environment=development
database=customer_database.db
batch_size=100
retry_count=3
minimum_age=18
maximum_age=100
minimum_rating=1
maximum_rating=5
allow_null_customer_id=false

```

```

In [36]: # =====
# STEP 15: PARSE CONFIGURATION FILE
# =====

config = {}

for line in raw_config.strip().split("\n"):

    if "=" in line:

```

```
        key, value =  
            line.split( "=",  
                1  
            )  
  
        config[key.strip()] = value.strip()  
  
print("Parsed Configuration:")  
  
for key, value in config.items():  
  
    print(  
        f"{key} = {value}"  
    )
```

Parsed Configuration:
source_system = customer_shopping_system
environment = development
database = customer_database.db
batch_size = 100
retry_count = 3
minimum_age = 18
maximum_age = 100
minimum_rating = 1
maximum_rating = 5
allow_null_customer_id = false

```
In [37]: # =====  
# STEP 16: ESTABLISH OPERATIONAL INGESTION CRITERIA  
# =====  
  
print("OPERATIONAL INGESTION CRITERIA")  
print("=" * 60)  
  
rules = [  
    "1. Customer ID must not be NULL.",  
    "2. Customer ID must be unique.",  
    "3. Customer name must not be NULL.",  
    "4. Email must not be NULL.",  
    "5. Email must follow a valid email format.",  
    "6. Age must be between 18 and 100.",  
    "7. Purchase amount must be greater than 0.",  
    "8. Review rating must be between 1 and 5.",  
    "9. Category must not be NULL.",  
    "10. Signup date must contain a valid date."  
]  
  
for rule in rules:  
    print(rule)
```

OPERATIONAL INGESTION CRITERIA

=====

1. Customer ID must not be NULL.
2. Customer ID must be unique.
3. Customer name must not be NULL.
4. Email must not be NULL.
5. Email must follow a valid email format.
6. Age must be between 18 and 100.
7. Purchase amount must be greater than 0.
8. Review rating must be between 1 and 5.
9. Category must not be NULL.
10. Signup date must contain a valid date.

```
In [38]: # =====
# STEP 17: VALIDATE INCOMING CUSTOMER RECORDS
# =====

def validate_customer(row):

    reasons = []

    # Customer ID validation
    if pd.isna(row["Customer ID"]):
        reasons.append(
            "Missing Customer ID"
        )

    # Name validation
    if pd.isna(row["Name"]):
        reasons.append(
            "Missing Name"
        )

    # Email validation
    if pd.isna(row["Email"]):

        reasons.append(
            "Missing Email"
        )

    elif not
        re.match( email_patt
ern,
str(row["Email"]))
):

        reasons.append(
            "Invalid Email Format"
        )

    # Age validation
    if pd.isna(row["Age"]):

        reasons.append( "Mis
sing Age"
        )

    elif (
        row["Age"] < 18
        or row["Age"] > 100
    )
```

```

        reasons.append( "Invalid Age"
                        )

# Purchase amount validation
if pd.isna(
    row["Purchase Amount (USD)"]
):

    reasons.append(
        "Missing Purchase Amount"
    )

elif (
    row["Purchase Amount (USD)"] <= 0
):

    reasons.append(
        "Invalid Purchase Amount"
    )

# Review rating
if pd.isna(
    row["Review Rating"]
):

    reasons.append(
        "Missing Review Rating"
    )

elif not (
    1 <= row["Review Rating"] <= 5
):

    reasons.append(
        "Invalid Review Rating"
    )

# Category
if pd.isna(
    row["Category"]
):

    reasons.append(
        "Missing Category"
    )

# Date validation
if pd.isna(
    row["Parsed Signup Date"]
):

    reasons.append(
        "Invalid Signup Date"
    )

return "; ".join(reasons)

```

```
In [39]: df["Rejection Reason"] =  
         df.apply( validate_customer,  
                   axis=1  
         )  
  
         df["Is Valid"] = (  
             df["Rejection Reason"] == ""  
         )  
  
         print(  
             "Validation rules applied successfully."  
         )  
  
         display(  
             df[  
                 [  
                     "Customer ID",  
                     "Name",  
                     "Email",  
                     "Is Valid",  
                     "Rejection Reason"  
                 ]  
             ].head(20)  
         )
```

Validation rules applied successfully.

	Customer ID	Name	Email	Is Valid	Rejection Reason
0	1	Allison Hill	donaldgarcia@example.net	True	
1	2	Abigail Shaffer	jpeterston@example.org	True	
2	3	Benjamin Stanley	tracie31@example.com	True	
3	4	Sarah Parsons	lydiatrujillo@example.com	True	
4	5	Zachary Hicks	camposmichelle@example.org	True	
5	6	Troy Liu	NaN	False	Missing Email
6	7	Shawn Arroyo	dennislisa@example.net	True	
7	8	Austin Smith	briannasmith@example.net	True	
8	9	Brent Jordan	ujenkins@example.org	True	
9	10	Savannah Garcia	novaksara@example.org	True	
10	11	NaN	smoore@example.org	False	Missing Name
11	12	Jennifer David	bethwilliams@example.org	True	
12	13	Daniel Baker	nicole35@example.com	True	
13	14	Douglas Clark	james48@example.com	True	
14	15	Jacob Tran	tanderson@example.com	True	
15	16	Norman Cook	vpham@example.org	True	
16	17	Diana May	ifrazier@example.com	True	
17	18	Kenneth Scott	zsmith@example.net	True	
18	19	Stephanie Gilbert	richard04@example.com	True	
19	20	Nicole Chambers	lindsayhernandez@example.org	True	

In [40]: *# Duplicate Customer IDs are also rejected*

```
duplicate_id_mask =  
    df.duplicated( subset=["Customer ID"],  
                  keep="first"  
    )  
  
df.loc[  
    duplicate_id_mask,  
    "Is Valid"  
] = False  
  
df.loc[  
    duplicate_id_mask,  
    "Rejection Reason"
```



```
] = df.loc[
    duplicate_id_mask,
    "Rejection Reason"
].apply(
    lambda x:
        (x + "; " if x else "")
        + "Duplicate Customer ID"
)

print(
    "Duplicate Customer ID validation completed."
)
```

Duplicate Customer ID validation completed.

```
In [41]: # =====
# STEP 18: SEPARATE VALID AND REJECTED RECORDS
# =====

clean_df = df[
    df["Is Valid"] == True
].copy()

rejected_df = df[
    df["Is Valid"] == False
].copy()

print(
    "Total Incoming Records:",
    len(df)
)

print(
    "Accepted Records:",
    len(clean_df)
)

print(
    "Rejected Records:",
    len(rejected_df)
)
```

Total Incoming Records: 501

Accepted Records: 490

Rejected Records: 11

```
In [42]: print("REJECTED RECORDS")
print("=" * 60)

display(
    rejected_df[
        [
            "Customer ID",
            "Name",
            "Age",
            "Email",
            "Purchase Amount (USD)",
            "Review Rating",
        ]
    ]
)
```

```
        "Signup Date",
        "Rejection Reason"
    ]
]
)
```

REJECTED RECORDS

=====

	Customer ID	Name	Age	Email	Purchase Amount (USD)	Review Rating	Signup Date	R
5	6	Troy Liu	66	NaN	1399.30	2.4	2024-08-03	
10	11	NaN	60	smoore@example.org	1548.41	4.9	2026-06-11	
20	21	Shannon McClure	26	zrobinson@example.net	190.91	1.2	2025-06-27	
25	26	Travis Marshall	45	invalid-email	912.91	4.8	2025-12-24	
30	31	Katherine Salas	33	abc@	489.33	3.3	2024-11-11	
35	36	Jennifer Hodges	46	gibsonleonard@example.com	250.80	1.9	wrong-date	
40	41	Paige Padilla	150	wallaceheather@example.org	202.89	2.7	2025-03-14	
45	46	Connie Brown	-5	tblankenship@example.net	110.67	5.0	2025-08-27	
50	51	Patricia Le	60	russellbeasley@example.com	-500.00	2.2	2024-02-16	
55	56	James Padilla	44	charles06@example.org	1870.07	7.0	2023-12-12	
500	1	Allison Hill	58	donaldgarcia@example.net	59.77	2.1	2024-10-03	C

```
C   C
```

```
In [43]: # Remove temporary parsed column if desired

clean_df.to_csv(
    "output/clean_customers.csv",
    index=False
)

rejected_df.to_csv(
    "output/rejected_customers.csv",
```

```
        index=False
    )

    print(
        "Accepted and rejected datasets saved successfully."
    )
```

Accepted and rejected datasets saved successfully.

```
In [44]: # =====
# STEP 19: GENERATE DATA QUALITY REPORT
# =====

quality_report =
    { "Total
      Records":
          len(df),

      "Records With Missing Values":
          int(df.isnull().any(axis=1).sum()),

      "Duplicate Records":
          int(df.duplicated().sum()),

      "Invalid Email Records":
          len(invalid_emails),

      "Invalid Date Records":
          len(invalid_dates),

      "Invalid Age Records":
          len(invalid_age),

      "Invalid Purchase Records":
          len(invalid_purchase),

      "Invalid Rating Records":
          len(invalid_rating),

      "Accepted Records":
          len(clean_df),

      "Rejected Records":
          len(rejected_df)
    }

quality_report_df =
    pd.DataFrame( quality_report.items(),
        columns=[
            "Quality Metric",
            "Count"
        ]
    )
```

	Quality Metric	Count
0	Total Records	501
1	Records With Missing Values	5
2	Duplicate Records	0
3	Invalid Email Records	2
4	Invalid Date Records	1
5	Invalid Age Records	2
6	Invalid Purchase Records	1
7	Invalid Rating Records	1
8	Accepted Records	490
9	Rejected Records	11

```
In [45]: # =====
# STEP 19: GENERATE DATA QUALITY REPORT
# =====

quality_report =
{ "Total
Records":
    len(df),

  "Records With Missing Values":
    int(df.isnull().any(axis=1).sum()),

  "Duplicate Records":
    int(df.duplicated().sum()),

  "Invalid Email Records":
    len(invalid_emails),

  "Invalid Date Records":
    len(invalid_dates),

  "Invalid Age Records":
    len(invalid_age),

  "Invalid Purchase Records":
    len(invalid_purchase),

  "Invalid Rating Records":
    len(invalid_rating),

  "Accepted Records":
    len(clean_df),

  "Rejected Records":
    len(rejected_df)
}

quality_report_df =
```

```

        columns=[
            "Quality Metric",
            "Count"
        ]
    )

    display(quality_report_df)

```

	Quality Metric	Count
0	Total Records	501
1	Records With Missing Values	5
2	Duplicate Records	0
3	Invalid Email Records	2
4	Invalid Date Records	1
5	Invalid Age Records	2
6	Invalid Purchase Records	1
7	Invalid Rating Records	1
8	Accepted Records	490
9	Rejected Records	11

```

In [46]: acceptance_rate = (
            len(clean_df) / len(df)
        ) * 100

        rejection_rate = (
            len(rejected_df) / len(df)
        ) * 100

        print(
            f"Acceptance Rate: {acceptance_rate:.2f}%"
        )

        print(
            f"Rejection Rate: {rejection_rate:.2f}%"
        )

```

Acceptance Rate: 97.80%
Rejection Rate: 2.20%

```

In [47]: # =====
# STEP 20: CREATE SQLITE RELATIONAL DATABASE
# =====

connection =
    sqlite3.connect( "customer_database.db
    )

clean_df.to_sql( "cust

```

```
        connection,
        if_exists="replace",
        index=False
    )

    print(
        "SQLite database connection established and 'customers' table created."
    )
```

SQLite database connection established and 'customers' table created.

```
In [48]: transaction_df.to_sql( "tran
        sactions",
        connection,
        if_exists="replace",
        index=False
    )

    print(
        "Transaction records stored in SQLite."
    )
```

Transaction records stored in SQLite.

```
In [49]: # =====
# STEP 21: INGEST DATA FROM RELATIONAL DATABASE
# =====

query = """
SELECT *
FROM customers
LIMIT 10;
"""

database_df =
    pd.read_sql_query( query,
        connection
    )

display(database_df)
```

	Customer ID	Name	Age	Gender	Email	Phone	
0	1	Allison Hill	58	Male	donaldgarcia@example.net	+1-219-560-0133	Robinson
1	2	Abigail Shaffer	26	Male	jpeterston@example.org	001-851-316-1559x40781	Maria
2	3	Benjamin Stanley	55	Female	tracie31@example.com	575-425-5341x928	Gray
3	4	Sarah Parsons	50	Male	lydiatrujillo@example.com	513-695-3767x242	Hurs
4	5	Zachary Hicks	44	Male	camposmichelle@example.org	+1-326-691-6697x8480	Danielch
5	7	Shawn Arroyo	31	Female	dennislisa@example.net	001-822-778-2489x63834	Lamm
6	8	Austin Smith	72	Female	briannasmith@example.net	+1-403-910-5183x4738	Port Tho
7	9	Brent Jordan	52	Male	ujenkins@example.org	+1-610-965-1333x87262	Port Sa
8	10	Savannah Garcia	36	Female	novaksara@example.org	(267)873-6026	Saram
9	12	Jennifer David	73	Male	bethwilliams@example.org	(799)385-4353x46247	S Christine

```
In [50]: query = """
SELECT COUNT(*) AS total_customers
FROM customers;
"""

result =
    pd.read_sql_query( query
    ,
    connection
    )
```

total_customers	
0	490

```
In [51]: query = """
SELECT
    AVG("Purchase Amount (USD)")
    AS average_purchase_amount
FROM customers;
"""

result =
    pd.read_sql_query( query
    ,
    connection
    )
```

average_purchase_amount	
0	1010.695673

```
In [52]: query = """
SELECT
    Category,
    COUNT(*) AS total_customers,
    ROUND(
        AVG("Purchase Amount (USD)"),
        2
    ) AS average_purchase
FROM customers
GROUP BY Category
ORDER BY total_customers DESC;
"""

category_result =
    pd.read_sql_query( query,
    connection
    )

display(category_result)
```

	Category	total_customers	average_purchase
0	Home	105	992.80
1	Accessories	103	945.18
2	Footwear	99	1007.57
3	Clothing	99	1050.37
4	Electronics	84	1070.32


```
In [53]: query = """
SELECT
    Category,
    ROUND(
        SUM("Purchase Amount (USD)"),
        2
    ) AS total_sales
FROM customers
GROUP BY Category
ORDER BY total_sales DESC;
"""

sales_result =
    pd.read_sql_query( query,
        connection
    )

display(sales_result)
```

	Category	total_sales
0	Home	104243.60
1	Clothing	103987.07
2	Footwear	99749.29
3	Accessories	97354.01
4	Electronics	89906.91

```
In [54]: query = """
SELECT
    "payment.method" AS payment_method,
    COUNT(*) AS total_transactions
FROM transactions
GROUP BY "payment.method"
ORDER BY total_transactions DESC;
"""

payment_result =
    pd.read_sql_query( query,
        connection
    )

display(payment_result)
```

	payment_method	total_transactions
0	Cash	65
1	Debit Card	53
2	Credit Card	46
3	UPI	36

```
In [55]: connection.close()

print(
    "SQLite database connection closed successfully."
)
```

SQLite database connection closed successfully.

```
In [56]: # =====
# STEP 22: VERIFY GENERATED OUTPUT FILES
# =====

print("DATA DIRECTORY")
print("=" * 50)

for file in os.listdir("data"):
    print(file)

print("\nOUTPUT DIRECTORY")
print("=" * 50)

for file in os.listdir("output"):
    print(file)
```

```
DATA DIRECTORY
=====
config.txt
customers.csv
transactions.json

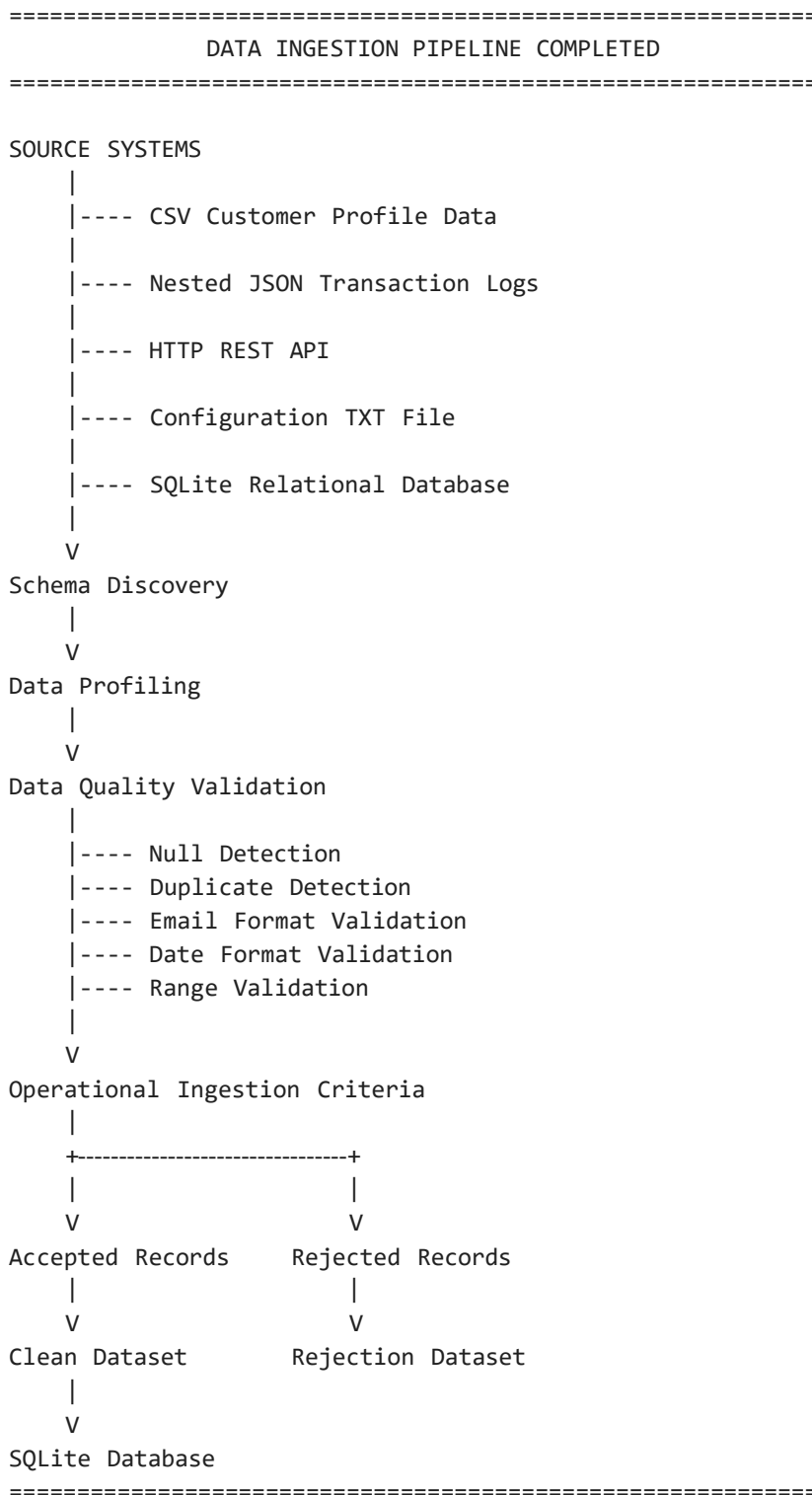
OUTPUT DIRECTORY
=====
api_users.csv
clean_customers.csv
flattened_transactions.csv
rejected_customers.csv
```

```
In [57]: # =====
# STEP 23: FINAL PIPELINE SUMMARY
# =====

print("""
=====
                        DATA INGESTION PIPELINE COMPLETED
=====

SOURCE SYSTEMS
|
|---- CSV Customer Profile Data
```

```
|
|---- Nested JSON Transaction Logs
|
|---- HTTP REST API
|
|---- Configuration TXT File
|
|---- SQLite Relational Database
|
V
Schema Discovery
|
V
Data Profiling
|
V
Data Quality Validation
|
|---- Null Detection
|---- Duplicate Detection
|---- Email Format Validation
|---- Date Format Validation
|---- Range Validation
|
V
Operational Ingestion Criteria
|
+-----+
|               |
V               V
Accepted Records   Rejected Records
|               |
V               V
Clean Dataset      Rejection Dataset
|
V
SQLite Database
=====
""" )
```



Learning Outcome

This practical provided hands-on experience in ingesting data from CSV, JSON, REST APIs, text files, and SQLite databases. It developed skills in synthetic data generation, schema discovery, data profiling, data quality validation, ingestion rules, and storing clean data in a relational database.

Conclusion

This practical successfully demonstrated data ingestion from CSV, nested JSON, REST API, text files, and SQLite databases using Python. Synthetic data was generated using Faker, and Pandas was used for schema discovery and data profiling. Data quality issues such as null values, duplicates, invalid formats, and invalid values were detected. Operational ingestion rules were applied to separate valid and rejected records, and clean data was successfully stored and queried using SQLite.